

### 1.1.1. Kiến trúc tổng quát của ứng dụng

Trong kiến trúc ứng dụng web nhiều tầng (multi-tier architecture), hệ thống thường bao gồm bốn thành phần chính: Client, Web Server, Application Server và Database Server. Việc hiểu rõ vai trò của từng thành phần là cơ sở để phân tích cách thức tấn công SQL Injection và các kỹ thuật vượt qua WAF.

**Client:** Là nơi người dùng trực tiếp tương tác với ứng dụng, thường là trình duyệt web hoặc ứng dụng di động (Web Client). Client gửi yêu cầu (HTTP/HTTPS request) đến hệ thống và nhận phản hồi (response) từ server. Đây cũng là nơi kẻ tấn công khởi phát các payload SQL Injection thông qua form nhập liệu, URL hoặc API. Ngoài ra, Client còn là nơi hiển thị kết quả dữ liệu, bao gồm cả nội dung tĩnh (logo, giao diện) và nội dung động (video, danh sách, thông tin người dùng).

**Web Server:** hay còn gọi là máy chủ Web, có nhiệm vụ lưu trữ nội dung trang web và xử lý các tài nguyên tĩnh như HTML, CSS, JS, hình ảnh. Đồng thời, Web Server

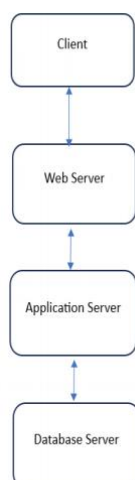
tiếp nhận yêu cầu từ Client và chuyển tiếp các yêu cầu động đến Application Server. Đây là vị trí thường triển khai WAF (Web Application Firewall) để lọc request độc hại. Nếu WAF bị bypass, payload SQL Injection sẽ đi thẳng xuống Application Server. Web Server giúp giảm tải cho Application Server bằng cách xử lý ngay các nội dung tĩnh, đồng thời đóng vai trò “cửa ngõ” cho toàn bộ hệ thống.

**Application Server:** là nơi xử lý logic nghiệp vụ. Nó đọc yêu cầu động từ Web Server, xác định hành động cần thực hiện dựa trên các quy tắc nghiệp vụ đã được lập trình sẵn, thực hiện xác thực, kiểm tra quyền truy cập, xử lý form và tạo truy vấn SQL gửi đến Database Server. Application Server chính là tầng biến dữ liệu đầu vào thành câu lệnh SQL. Nếu dữ liệu không được kiểm tra đúng cách, payload SQL Injection sẽ được Application Server gửi nguyên vẹn xuống Database Server.

**Database Server:** là nơi lưu trữ và quản lý dữ liệu của ứng dụng. Nó nhận và thực thi các câu lệnh SQL từ Application Server, bao gồm các thao tác như SELECT, INSERT, UPDATE, DELETE. Đây là mục tiêu cuối cùng của tấn công SQL Injection. Nếu payload vượt qua WAF và Application Server, nó sẽ được thực thi trực tiếp trên Database Server, gây rò rỉ dữ liệu, thay đổi dữ liệu hoặc thậm chí chiếm quyền hệ thống.

Database Server thường được bảo vệ bằng các cơ chế phân quyền, mã hóa dữ liệu, và giám sát truy vấn để giảm thiểu rủi ro.

Có thể được vẽ như sau:



Hình 1.1: Mô hình kiến trúc 4 tầng (4-Tier Architecture) và luồng xử lý truy vấn dữ liệu truyền thống.

Sơ đồ trên được thực hiện như sau Client -> Web Server -> Application Server -> Database Server -> Application Server -> Web Server -> Client

### 1.1.2. Các mối đe dọa bảo mật phổ biến đối với web

Cross-Site Scripting (XSS): là một lỗ hổng bảo mật phổ biến trong các ứng dụng web, cho phép kẻ tấn công chèn mã độc (thường là JavaScript) vào trang web. Khi người dùng khác truy cập trang đó, đoạn mã sẽ được trình duyệt thực thi, gây ra mất dữ liệu

liệu quan trọng hay mất kiểm soát thiết bị truy xuất.

Cross-Site Request Forgery (CSRF): là dạng tấn công nhằm đánh lừa người dùng truy cập vào trang web khác trên trang web hợp lệ từ đó kẻ tấn công thực hiện, thao túng, thay đổi tài khoản để đánh cắp tài sản dữ liệu trên danh nghĩa hợp pháp người dùng ban đầu mà Server không nhận ra đó không phải người dùng thật.

Tấn công chèn mã SQL (SQL Injection): là một dạng kỹ thuật cho phép kẻ tấn công chèn mã SQL vào dữ liệu để gửi yêu cầu đến máy chủ nhằm bỏ qua xác thực người dùng để đánh cắp thông tin nhạy cảm, thay đổi nội dung hoặc truy cập nội dung không được công khai. Dạng tấn công này thường gặp ở các ứng dụng Web, các trang có kết nối đến cơ sở dữ liệu.

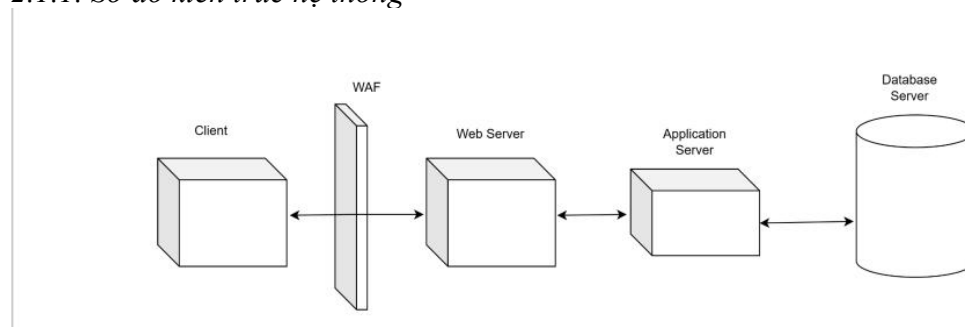
Có 2 nguyên nhân của lỗ hổng trong ứng dụng cho phép thực hiện tấn công chèn mã SQL:

- Dữ liệu đầu vào từ người dùng hoặc từ các nguồn khác không được kiểm tra hoặc kiểm tra không kỹ lưỡng.
- Sử dụng các câu lệnh SQL động trong ứng dụng, trong đó có thao tác ghép nối dữ liệu người dùng với mã SQL gốc.

## CHƯƠNG 2: THIẾT KẾ VÀ TRIỂN KHAI THỰC NGHIỆM

### 2.1 Mô Hình tổng thể hệ thống thực hiện

#### 2.1.1. Sơ đồ kiến trúc hệ thống



Hình 2.1: Sơ đồ kiến trúc hệ thống thực nghiệm tích hợp WAF ModSecurity.

#### 2.1.2. Mô tả chức năng từng thành phần

**Client:** Hoạt động phía trước WAF và Web Server là máy tính hay thiết bị đầu cuối là nơi Client gửi yêu cầu và nhận yêu cầu từ Web Server. Client thường là trang web hoặc ứng dụng tương tác với người dùng. Là nơi tương tác nhiều với người dùng cũng là nơi dễ thực hiện tấn công nhất nên cần phải rà soát nghiêm ngặt nhất có thể.

Gửi và nhận yêu cầu: Khi Client gửi yêu cầu như truy cập vào 1 trang URL hay đăng nhập vào web thì các thông tin này sẽ được gửi đến Database Server sau đó Client nhận yêu cầu phản hồi từ Web Server và hiển thị thông tin cho người dùng trên trang web đó

**Web Application Firewall:** Hoạt động ở tầng ứng dụng (Lớp 7 theo mô hình OSI) được đặt ở phía trước - - Web Server có nhiệm vụ giám sát, lọc và chặn các yêu cầu HTTP/HTTPS độc hại trước khi các yêu cầu đi vào ứng dụng web và ngăn chặn

mọi dữ liệu trái phép rời khỏi ứng dụng. Bảo vệ dữ liệu quan trọng và thông tin cá nhân: WAF ngăn chặn các cuộc tấn công có thể dẫn đến việc đánh cắp hoặc rò rỉ thông tin nhạy cảm, bảo vệ dữ liệu người dùng, giao dịch tài chính và thông tin kinh doanh quan trọng.

Giúp ngăn chặn các cuộc tấn công phổ biến như SQL Injection chặn việc chèn mã SQL vào form hoặc URL. Cross-Site Scripting (XSS) ngăn chặn chèn mã JavaScript độc hại. Cross-Site Request Forgery (CSRF) ngăn chặn giả mạo yêu cầu từ người dùng.

**Web Server:** Thường được đặt ở phía trước application là nơi tiếp nhận và phản hồi yêu cầu của Client thông qua giao thức HTTP/HTTPS để thực hiện các tài nguyên tĩnh và chuyển tài nguyên động đến Application Server để xử lý.

Thực hiện các tài nguyên tĩnh như hình ảnh văn bản, video,.. của HTML, CSS và JavaScript. Tiếp nhận các tài nguyên động khi người dùng đăng nhập, gửi form hoặc thanh toán thì sẽ được gửi tới Application Server để xử lý sau đó nhận phản hồi từ Application Server và chuyển tiếp đến Client

Được ứng dụng thường là tạo phiên cookie khi người dùng truy cập trang web lần đầu tiên và giữ cookie này khả dụng trong suốt quá trình người dùng tương tác với trang web.

Trong mô hình truyền thống (Hình 1.1), các request đi trực tiếp từ Client đến Web Server. Trong mô hình thực nghiệm bảo mật (Chương 2), WAF sẽ được triển khai đứng trước Web Server để lọc các request độc hại.

**Application Server:** Là một thành phần trung gian trong kiến trúc hệ thống web, nằm giữa Web Server và Database Server. Nó cung cấp môi trường để chạy các ứng dụng web động, xử lý logic nghiệp vụ phức tạp, quản lý bảo mật, và giao tiếp với cách hệ thống khác.

Xử lý logic nghiệp vụ (Business Logic): Chịu trách nhiệm xử lý các quy tắc và logic phức tạp của ứng dụng cụ thể như: đăng nhập vào ứng dụng trên web chỉ cho phép người dùng có trong dữ liệu thì mới có thể đăng nhập, truy cập vào ứng dụng web để chuyển tiền ghi nhận lại lịch sử giao dịch để phục vụ thanh toán,..

Quản lý bảo mật (Security Management): Chịu trách nhiệm đảm bảo an toàn cho ứng dụng web bằng cách kiểm soát truy cập, xác thực người dùng, và bảo vệ dữ liệu khỏi các mối đe dọa như: Xác thực người dùng xem có đúng người để truy cập vào các trang riêng tư, kiểm soát quyền chỉ cho phép người dùng có quyền hạn mới có thể kiểm soát và thay đổi quản trị dữ liệu còn người dùng thông thường chỉ có thể xem nội dung,...

Thực hiện Role-Based Access Control (RBAC): kiểm soát quyền truy cập vào URL hoặc chức năng dựa trên vai trò người dùng (admin, editor, viewer) có thể hiểu chỉ có quản trị mới có thể thấy URL ẩn và thay đổi các chức năng của hệ thống, editor có thể thấy URL ẩn nhưng không thể thay đổi nội dung chỉ dựa trên chức năng có sẵn đó để tạo web phù hợp với mục tiêu đề ra còn viewer thì không thể thấy URL ẩn đó chỉ thấy trang mặc định của web dù có biết và cố tình truy cập thì sẽ không thể thấy trang ẩn đó

**Database Server:** Là nơi cuối cùng của mô hình chứa tất cả dữ liệu của web tiếp nhận SQL từ Application Server xử lý truy vấn và trả về kết quả gửi đến cho Client đảm bảo tính toàn vẹn, bảo mật. Database Server có thể cung cấp nhiều tài khoản khác nhau với các vai trò khác nhau.

Chẳng hạn như khi thực hiện lệnh SQL để lấy dữ liệu nếu người không có quyền hạn thì chỉ có thể xem nội dung không thể chỉnh sửa hay xóa. Còn với những người có quyền hạn cao hơn thì có thể thêm xóa sửa nội dung. Điều này giúp hạn chế việc điều này làm giảm thiểu tác động của nhiều lỗ hổng tấn công SQL injection tránh mất

mát dữ liệu nhạy cảm.

## 2.4 Cấu hình Firewall (WAF)

### 2.4.1 Lựa chọn WAF cho thực nghiệm

Nhóm chọn OWASP ModSecurity CRS trên Nginx cho cấu hình WAF với các tiêu chí

lựa chọn WAF cho thực nghiệm thường có những điểm sau:35

#### **Khả năng bảo mật:**

Có bộ quy tắc chuẩn (OWASP CRS) để chống SQL Injection, XSS, LFI/RFI, CSRF.

Hỗ trợ cập nhật rule thường xuyên.

Cho phép tùy chỉnh rule để phù hợp với ứng dụng để có thể test bypass bằng SQL Injection, XSS, LFI/RFI, CSRF cho mục đích nghiên cứu.

#### **Tính dễ triển khai và quản lý:**

- Dễ dàng chạy bằng Docker, tích hợp với Nginx.
- Có tài liệu rõ ràng, cộng đồng hỗ trợ mạnh.
- Quản lý qua file cấu hình và log.

#### **Hiệu suất và khả năng mở rộng:**

- Độ trễ thấp khi xử lý request.
- Có thể mở rộng khi ứng dụng tăng tải.
- Tối ưu tài nguyên hệ thống.

#### **Chi phí:**

- Miễn phí (mã nguồn mở).
- Chỉ tốn chi phí hạ tầng vận hành.- Phù hợp cho môi trường thử nghiệm/lab.

### 2.4.2 Cài đặt và cấu hình WAF:

Trên Visual Studio Code tại file docker-compose.yml đã có sẵn code của db server và web-app ta sẽ thêm cấu hình WAF ở cuối code đó.

waf:

image: owasp/modsecurity-crs:nginx

ports:

- "80:8080"

environment:

- MODSEC\_RULE\_ENGINE=On
- BACKEND=http://web-app:8000

volumes:

- ./my\_sqli\_rules.conf:/etc/modsecurity.d/owasp-crs/rules/RESPONSE-999-EXCLUSIONS.conf36

depends\_on:

- web-app

- Waf: Đây là tên của service (dịch vụ) trong Docker Compose. Ở đây service được đặt

tên là waf (Web Application Firewall).

- image: owasp/modsecurity-crs:nginx

Chạy container Nginx có tích hợp ModSecurity và bộ luật OWASP CRS.

- ports: - "80:8080"

Map cổng 8080 trong container ra cổng 80 trên máy host. Người dùng truy cập qua http://localhost:80.

- environment:

+ MODSEC\_RULE\_ENGINE=On: bật engine ModSecurity để kiểm tra request.

+ BACKEND=http://web-app:8000: chỉ định backend (ứng dụng web thật) để

Nginx reverse proxy.

- volumes:

+./my\_sqli\_rules.conf:/etc/modsecurity.d/owasp-crs/rules/RESPONSE-999-EXCLUSIONS.conf

Mount file my\_sqli\_rules.conf từ máy host vào container, ghi đè file cấu hình exclusions của CRS. Đây là nơi tùy chỉnh rule.

Rule được ghi đè tại tệp my\_sqli\_rules.conf :

SecAction

"id:900110,phase:1,nolog,pass,t:none,setvar:tx.inbound\_anomaly\_score\_threshold=20

" (1)

SecRuleUpdateActionById 942100 "pass,log" (2)

SecRuleUpdateActionById 942160 "pass,log" (3)

SecRuleUpdateActionById 949110 "pass,log" (4)

SecRule ARGS "@rx --" "id:10001,phase:2,log,deny,status:403,msg:'SQL Comment Detected - Immediate Block',setvar:tx.anomaly\_score+=20" (5)

Rule (1):

+ id:900110: mã định danh cho rule này.

+ phase:1: chạy ở giai đoạn đầu (request headers).37

+ nolog, pass: không ghi log, cho qua.

+ setvar:tx.inbound\_anomaly\_score\_threshold=20: đặt ngưỡng anomaly score inbound lên 20.

Nghĩa là: chỉ khi tổng điểm vi phạm  $\geq 20$  thì mới bị chặn. Mặc định CRS thường đặt ngưỡng thấp hơn (5 hoặc 10).

Rule (2) và (3):

+ 942100, 942160: đây là các rule trong nhóm SQL Injection Detection của CRS.

+ pass,log: thay đổi hành động mặc định thành chỉ ghi log và cho qua request.

Nghĩa là: nếu phát hiện pattern SQLi theo các rule này, hệ thống sẽ không chặn ngay, chỉ log lại để theo dõi.

Rule (4):

+ 949110: rule tổng hợp anomaly score, thường quyết định chặn khi vượt ngưỡng.

+ pass,log: vô hiệu hóa hành động chặn, chỉ log.

Nghĩa là: rule này không còn “quyền lực” chặn nữa, giúp giảm false positive

Rule (5):

+ARGS "@rx --": kiểm tra tất cả tham số request, nếu có chuỗi -- (regex match).

+id:10001: mã định danh riêng cho rule này.

+phase:2: chạy ở giai đoạn phân tích request body.

+log, deny, status:403: ghi log, chặn request, trả về HTTP 403.

+msg:'SQL Comment Detected - Immediate Block': thông báo khi chặn.

+setvar:tx.anomaly\_score+=20: cộng thêm 20 điểm anomaly để chắc chắn vượt ngưỡng.

Nghĩa là: nếu phát hiện dấu comment SQL (--), coi là nguy hiểm rõ ràng → tăng ngưỡng lên 20 và chặn ngay lập tức.

- depends\_on:

+ web-app

Đảm bảo container WAF chỉ khởi động sau khi container web-app đã chạy.38

#### 2.4.3. Kiểm tra hoạt động của WAF

Để kiểm tra xem waf có hoạt động đúng, ta sẽ kiểm thử với các tình huống tấn

công phổ biến:

- SQL Injection: nhập payload ' OR '1'='1-- vào thanh email ở phần body.
- Bypass thử nghiệm: thay đổi ký tự comment từ -- sang # để kiểm tra khả năng phát hiện.

### **Kết quả tổng quát:**

- Các payload SQL Injection có chứa dấu -- bị WAF phát hiện và chặn trả về 403 Forbidden.
- Payload SQL Injection có chứa dấu # thì truy cập thành công (200 OK).

### **Ý nghĩa:**

- WAF đã chứng minh khả năng phát hiện và ngăn chặn tấn công bằng SQL Injection.
- Đồng thời, kiểm thử cũng cho thấy khả năng bypass nếu rule chưa bao phủ hết các dạng payload, từ đó nhấn mạnh tầm quan trọng của việc tinh chỉnh và cập nhật rule thường xuyên.

### **3.1 Kịch bản SQL Injection bằng payloads cơ bản**

Trong phần này, nhóm tiến hành kiểm thử lỗ hổng **SQL Injection** trên chức năng đăng nhập của hệ thống, nhằm đánh giá hiệu quả của cơ chế bảo vệ khi có và không có

Web Application Firewall (WAF).

Thông tin tài khoản hợp lệ được sử dụng để đối chiếu như sau:

Tài khoản admin: admin@gmail.com

Mật khẩu: 123456



Hình 3.1: Web đăng nhập bình thường

Kết quả cho thấy hệ thống cho phép đăng nhập thành công.

### **Trường hợp 1: Tấn công khi đã triển khai WAF**

Sau khi tích hợp WAF (ModSecurity với OWASP CRS), nhóm thực hiện lại payload tấn công với payload như sau:

SQL: admin@gmail.com'or '1'='1--

Mật khẩu: nhập bất kì

Kết quả nhận được là phản hồi lỗi:

## **403 Forbidden**

nginx

Hình 3.2: Giao diện WAF chặn request trả về lỗi 403 Forbidden.

Điều này cho thấy WAF đã phát hiện request chứa dấu hiệu của SQL Injection và chủ động chặn truy cập trước khi request được chuyển đến backend.

### **Trường hợp 2: Tìm thấy điểm yếu của WAF và Bypass WAF bằng biến thể payload**

Mặc dù WAF có khả năng phát hiện các mẫu SQL Injection phổ biến, tuy nhiên trong một số trường hợp, việc cấu hình rule chưa đủ chặt chẽ có thể tạo ra điểm yếu.<sup>43</sup>

Cụ thể, nhóm nhận thấy WAF chủ yếu chặn các payload sử dụng cú pháp comment --. Do đó, một biến thể payload được sử dụng:

SQL: admin@gmail.com'or '1 '='1#

Mật khẩu: nhập bất kì



### Hình 3.3: Web bypass thành công

Trong đó, ký tự # cũng đóng vai trò comment trong SQL nhưng không bị WAF phát hiện trong cấu hình hiện tại.

Kết quả thực nghiệm cho thấy payload này vẫn có thể bypass WAF và đăng nhập thành công, tương tự như trường hợp không có WAF.

### 3.1.1. Phân tích log WAF và thông báo chặn:

Ở trường hợp 2 bypass không thành công khi waf thấy và chặn được thì log sẽ ghi nhận như sau:

Tại nơi được tô đậm đó là yêu cầu từ chối của WAF phát hiện tấn công:

[illegible]

Nằm ở cuối dòng lênh:

```
"Operator 'Rx' with parameter '--' against variable 'ARGS:email' (Value: 'admin@gmail.com' or '1='1--')", "reference": "026,2v997,28", "ruleId": "10001", "file": "/etc/modsecurity.d/owasp-crs/rules/RESPONSE-999-EXCLUSIONS.conf", "lineNumber": "11", "data": {"severity": "0", "ver": "", "rev": "", "tags": ["modsecurity", "maturity": "0", "accuracy": "0"]}}
```

### Hình 3.4: Chi tiết Audit Log ModSecurity kích hoạt Rule 10001

- variable ARGS:email: rule kiểm tra tham số email trong body request.
- Value: giá trị nhập vào là admin@gmail.com 'or' '1'='1--.
- ruleId 10001: đây là rule đã được chỉnh sửa.
- msg "SQL Comment Detected - Immediate Block": thông báo rằng rule đã phát hiện dấu hiệu comment SQL (--) và chặn ngay lập tức.<sup>44</sup>
- http\_code 403: request bị từ chối, trả về lỗi Forbidden.

Ở trường hợp 3 khi bypass thành công khi tìm thấy lỗi hỏng trong waf và log cụ thể như sau:

```
172.18.0.1 - - [25/Mar/2026:16:44:43 +0000] "GET /admin-panel/ HTTP/1.1" 200
807 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Edg/146.0.0.0" "-"
```

Client (IP 172.18.0.1) gửi GET tới /admin-panel/. Server trả về 200 OK truy cập thành

công vào trang quản trị.

Đây là dấu hiệu cho thấy request đã bypass WAF (không bị chặn bởi rule nào).